

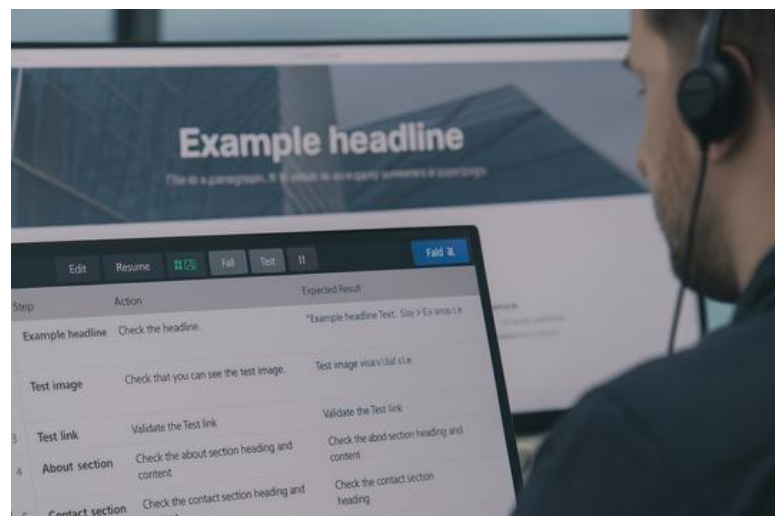
MATERIAL COMPLEMENTARIO – UNIDAD 8

Calidad: Análisis de Resultados y Acciones Correctivas

1. Propósito del Apunte

Este apunte tiene la intención de **profundizar la mirada sobre la calidad** luego de que se ejecutan las pruebas. Así como en los modelos de desarrollo la clave está en entender qué filosofía organiza cada modelo, en esta unidad el foco está en entender **qué filosofía organiza la calidad** cuando ya existe evidencia concreta de cómo se comporta el software. Las pruebas no son el final del camino, sino el punto donde comienza una etapa distinta: la etapa del **análisis**, donde la evidencia se transforma en comprensión y, sobre todo, en **decisiones que mejoran el producto**.

Este documento no introduce herramientas nuevas, sino que intenta ofrecer un marco conceptual para leer e interpretar los resultados que deja el testing, y para ordenar las decisiones que siguen en términos de corrección, priorización y mejora continua.



2. Comprender la Naturaleza del Defecto

Un resultado de prueba nunca habla por sí solo. Lo que vemos —un error en pantalla, un cálculo incorrecto, una validación ausente— es apenas la superficie de un problema mucho más profundo.

Por eso, el primer paso en la calidad no es “arreglar”, sino **comprender**.

En esta unidad es fundamental distinguir entre **error**, **defecto** y **falla**.

Un error es humano: alguien tomó una decisión incorrecta, redactó mal un requerimiento, dejó pasar un escenario o codificó una condición equivocada. El defecto es el producto directo de ese error en el código o la configuración. Y la falla es la manifestación visible de ese defecto al ejecutar el software.

Este enfoque es central: si solo se “arregla lo que se ve”, se corrige la falla, pero permanece intacto el error que la originó. La calidad comienza cuando se entiende que el objetivo no es tapar el síntoma, sino intervenir en la raíz del problema.

Asimismo, comprender la naturaleza del defecto implica comprender su **categoría**. Un error de validación, un cálculo lógico incorrecto, un problema de interfaz o una falla de integración no son equivalentes. Cada uno involucra capacidades y responsabilidades distintas dentro del equipo, y afecta también qué tipo de pruebas deben realizarse después de corregirlo.

3. Severidad y Prioridad: Dos Miradas Complementarias

En el análisis de post-testing aparecen dos preguntas diferentes, que muchas veces se confunden:

- ¿Qué tan grave es lo que está pasando desde el punto de vista técnico?
- ¿Qué tan urgente es solucionarlo desde el punto de vista del negocio?

La primera pregunta se responde con la **severidad**, la segunda con la **prioridad**.

La severidad se concentra en el funcionamiento interno: cuánto se ve afectada la lógica del sistema o qué tan comprometidas están sus funciones principales.

La prioridad, en cambio, se concentra en el contexto real: la operación diaria, los usuarios, la imagen institucional, el dinero, los tiempos.

Es perfectamente posible —y común— que un defecto tenga **baja severidad y alta prioridad**. Por ejemplo, un certificado con acentos mal impresos no bloquea el funcionamiento del sistema, pero es inaceptable para una institución.

También puede darse el caso inverso: una falla grave en un módulo que no se utiliza en la versión actual puede ser severa, pero no prioritaria.

Entender estas combinaciones es parte de la maduración profesional. No existe calidad sin esta capacidad de análisis diferenciado.

4. La Causa Raíz: Mirar Debajo de la Superficie

Un defecto resuelto no es necesariamente un problema resuelto.

La experiencia real muestra que muchos equipos corrigen fallas pero no corrigen **los motivos por los cuales esas fallas ocurren**. Como resultado, el mismo problema reaparece una y otra vez, disfrazado de diferentes formas.

El análisis de causa raíz es la herramienta que permite romper este ciclo. No importa si se usa la técnica de los 5 *Whys* o un diagrama de Ishikawa: lo importante es asumir que el defecto no fue “un descuido”, sino un síntoma de un proceso que falló.

La causa raíz puede estar en un requerimiento mal definido, una interpretación ambigua, una falta de casos de prueba representativos, un error en la relación entre equipos, o incluso en la cultura de trabajo del proyecto.

Lo central es comprender que, desde la perspectiva de la calidad, **el hallazgo es una oportunidad de aprendizaje**, no una acusación.

5. La Acción Correctiva como Compromiso Profesional

Corregir un defecto no consiste únicamente en modificar código. Una acción correctiva profesional establece con claridad:

- qué debe ser cambiado,
- cómo se sabrá que fue cambiado correctamente,
- y qué partes del sistema podrían verse afectadas indirectamente.

Una buena acción correctiva es específica, fundamentada y verificable. No deja lugar a interpretaciones ambiguas y convierte el análisis en un plan concreto de mejora.

Una mala acción correctiva, en cambio, genera inestabilidad, inconsistencias y, en el peor de los casos, nuevas fallas. Lo que parecía una simple corrección puede convertirse en un detonante de regresiones si no se piensa con cuidado.

6. Re-testing y Regresión: Dos Formas de Verificar

Cuando una corrección se implementa, el trabajo no termina. De hecho, recién empieza otra etapa crítica: **verificar que esté realmente arreglado y asegurar que nada más se haya roto**.

El **re-testing** está enfocado en el caso original. Es un test puntual, directo, casi quirúrgico: “lo que estaba mal ahora funciona como debe”.

La **regresión**, en cambio, es una mirada ampliada.

El propósito no es confirmar el arreglo, sino asegurar que el cambio no haya impactado en módulos relacionados, cálculos asociados, flujos alternativos o componentes dependientes.

Esta distinción es importante porque los desarrollos modernos —sobre todo los basados en microservicios, frameworks, o dependencias internas complejas— pueden alterar comportamientos que no tienen relación visible con el defecto inicial.

Una corrección que no se valida con regresión es una invitación abierta a que surjan nuevos errores en producción.

7. La Calidad como Transformación

La unidad propone una idea que vale la pena destacar y desarrollar: **la calidad no es encontrar errores; es mejorar el sistema a partir de la evidencia.**

Cuando los estudiantes analizan evidencia, clasifican defectos, asignan severidad y prioridad, hacen causa raíz y proponen acciones correctivas, están aprendiendo una mirada profesional que excede la técnica del testing. Están aprendiendo una forma de pensar: **una forma de transformar problemas en mejoras estructurales.**

La calidad, entendida así, no es una etapa del proceso. Es un modo de organizar la experiencia, de aprender de los defectos, de anticipar riesgos y de acompañar al equipo hacia un producto más robusto, más estable y más coherente con las necesidades reales del usuario. En última instancia, este apunte busca que puedan ver la unidad no solo como un conjunto de técnicas, sino como un marco conceptual para entender qué significa hacer calidad en el mundo real.